FOLIOQ

Multimodal Document Intelligence Platform

1. The Vision & Core Problem

Most AI document tools available today struggle with real-world business PDFs. Traditional systems often treat every file as a plain string of text, regardless of document type - whether it is an annual report, an Operations Research exam paper, an engineering blueprint, or a multi-column document.

They extract text, split it into arbitrary 500-token chunks, and send those chunks to an LLM, often producing answers without reliable source verification.

I designed and built FolioQ around a clear architectural idea: Evidence Over Inference.

FolioQ treats documents as visual, structured artefacts rather than merely text on a page. It reads complex, multi-page PDFs, processes them asynchronously through a multimodal pipeline, semantically indexes both content and exact layout structures, and provides precise visual page citations alongside sub-second streaming answers to user questions.

2. The Ingestion Pipeline & Multimodal Processing

This is achieved through a decoupled, asynchronous ingestion pipeline, with a background worker queue powered by Redis and BullMQ.

The main API does not block when a document is uploaded. Instead, it delegates processing to a throughput-oriented, fault-tolerant pipeline consisting of five stages.

The pipeline converts every PDF page into a high-resolution image in parallel and extracts native glyphs together with normalized bounding-box coordinates.

For complex pages containing financial tables, diagrams, or charts, the engine activates multimodal vision models (Gemini Flash) to extract visual hierarchy, table schemas, and data trends that traditional OCR may miss.

Rather than enforcing a fixed word-count limit, FolioQ chunks content along natural document boundaries, keeping entire tables, headings, and semantic sections intact.

Each chunk is then embedded into a high-dimensional vector space and stored in MongoDB with its exact page number and spatial coordinates.

3. Hybrid Retrieval & Sub-Second Answering

Before matching a user's query, FolioQ can process context-dependent questions such as:

"Is it for 7 subjects or 8 subjects?"

The rewriter resolves pronouns and conversation history into a standalone retrieval intent.

FolioQ then uses Hybrid Retrieval, combining two complementary approaches:

Dense Vector Similarity: Identifies high-level conceptual matches and semantically related content.

Sparse Lexical Similarity: Finds exact identifiers, course codes, numeric constants, and other precise terms.

The most relevant chunks are reranked and combined into an authoritative context envelope alongside the corresponding source-page images.

This context is sent to a high-speed LLM gateway through Groq, with time to first token kept below one second.

The model is constrained by the available evidence:

- It must not answer beyond the retrieved context.
- It may quote only literal numbers found in the source.
- It formats complex tables clearly.
- It references each statement with a document and page marker in the format.

5. The Verification & Grounding Guarantee

Users do not have to rely on the model's word alone. FolioQ is not a generic chat wrapper.

Answers pass through a verification process that checks whether claims match the retrieved text.

Citations in the workspace are clickable. Selecting a citation pill or a table-row citation opens the high-resolution source page in the built-in inspector, with the relevant bounding box highlighted over the source paragraph or table.

When an answer contains tabular data, a custom parser recognizes the table structure and renders it as an interactive table featuring:

- Sticky headers.
- Automatic column alignment.
- Structured table formatting.
- Citation references for individual rows.
- Quick CSV export for analysis in tools such as Microsoft Excel.

This ensures that users can move directly from an AI-generated answer to the original visual evidence supporting it.

6. Performance & System Resilience

FolioQ is structured as an enterprise-grade monorepo containing three major components:

1. API Server: Handles document uploads, user queries, and client communication.
2. Background Worker: Processes document ingestion and indexing asynchronously.
3. Client Application: Provides the document workspace, streaming answers, interactive tables, and visual source inspection.

Its storage layer supports directory-backed previews and on-demand rendering.

If an intermediate preview image is not present on disk, the system dynamically re-renders and serves the page without disrupting the user.

The system is designed to be self-contained, high-performance, and resilient to vendor downtime.

Its ultimate goal is to transform chaotic, unstructured document archives into authoritative, verifiable knowledge.